Universidad de La Sabana
Facultad de Ingeniería

Felix Mohr & Sergio Amortegui
2026.1

Fundamentos de Inteligencia Artificial

- Sheet 6-

Environment & Rules. Use Python with only `numpy`. Do not use global random state; all randomness must come from a provided `numpy.random.Generator` named `rng`. Unless explicitly stated otherwise, functions that sample must accept a seed or `rng`. Keep the function signatures from the provided `.py` templates unchanged, as they will be tested automatically on Gradescope. You will only see public test outputs.

Files to implement. `lake_mdp.py`, `policies.py`, `utility_analyzer.py`, `run.py`. Do not rename files or functions.

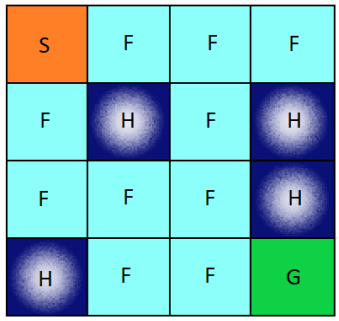
Environment Considerations: Lake MDP. We consider the *Lake* environment on an $m \times n$ grid. Each cell is one of

`S` (unique start), `F` (free), `H` (hole, terminal), `G` (goal, terminal).

At each non-terminal step the agent chooses an action in $\{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}\}$. With probability 0.8 the intended move is executed; with probability 0.1 each, the move slips to the two lateral directions if the state is not an absorbing one. Bumping against borders keeps the agent in place.

Rewards depend *only* on the state *entered*: a constant step reward in `F` ($+0.1$), a fixed reward in `H` (-1.0), and a fixed reward in `G` ($+1$).

Upon entering any terminal cell, the process transitions to a synthetic absorbing state $s_\perp$ with only action $\perp$ and reward 0 thereafter.



Exercise 1 (Exercise 1: Implement the LakeMDP class – 3 Points) Complete the class `LakeMDP` in `lake_mdp.py` so that it realises the dynamics specified above.

Requirements: d

- Rewards are given on *entry* into the next state.
- After entering any terminal cell, the next state is the absorbing state $s_\perp$ with only action $\perp$ and reward 0 forever.
- Each transition from a non-terminal state must return a valid probability distribution over next states.

Exercise 2 (Exercise 2: Implement the Policies – 2 Points) In `src/policies.py` implement:

1. `RandomPolicy` that selects uniformly at random among the legal actions in a state.
2. `CustomPolicy` that you design (deterministic or stochastic) and that *does not learn or adapt* but is intended to perform *somewhat better than* the random policy on the provided Lake map. The policy may consult the MDP for legal actions and map layout but must be a fixed mapping from state to an action.

Exercise 3 (Exercise 3: Implement the Utility Analyzer – 3 Points) In `utility_analyzer.py` complete the class `UtilityAnalyzer` with two public methods:

1. `run_trial(policy, seed)`: starting at the MDP's start state, generate a trial $\tau = s_0, a_0, r_1, s_1, \ldots$ until a terminal cell is entered (then $s_\perp$) or a given step limit is reached; return the discounted utility $U(\tau) = \sum_{t \geq 0} \gamma^t r(S_t)$, the trial length, and which terminal (goal/hole/none) was reached.
2. `evaluate(policy, n_trials, base_seed)`: run n_{trials} independent trials (seeded as `base_seed + i`) and return a summary with at least the mean utility, utility variance, probability of reaching the goal, probability of falling into a hole, and the mean trial length.

Exercise 4 (Exercise 4: Implement the Runner – 2 Points) Complete `run.py` to:

1. Instantiate the Lake MDP and both policies.
2. Evaluate `RandomPolicy` and `CustomPolicy` for $\gamma \in \{0.5, 0.9, 1.0\}$ with a fixed number of trials (10) and seeds (base: 42).
3. Return a JSON report that includes the summaries for both policies and indicates which policy attains the higher mean utility (tie-breaker: smaller variance).

Exercise 5 (Practice: Understanding the Lake MDP) These problems are not graded on Gradescope but will help you understand the concepts and will be discussed in class.

1. For the 4×4 instance (start at top-left, goal at bottom-right, at least one hole), formalize the MDP by explicitly specifying the set of states, the action sets, the transition probabilities, and the reward function (use the absorbing-state convention).
2. Argue whether there exists a policy that reaches the goal with certainty without any chance of falling into a hole. Justify concisely.
3. The utility of a run is $U(\tau) = \sum_{t=0}^{\infty} \gamma^t r(S_t)$ with $\gamma \in (0, 1]$. Discuss briefly how different choices of γ would influence reasonable behaviour in the Lake MDP.